The software product that you're working on ends up getting compiled and deployed at some point. Before that happens, your team is working on the *source*, or the code that comprises the functionality you want to ship to your consumers. Because you likely have more than one person working on this source, you need to *manage* it somewhere to ensure that changes aren't getting overwritten, good collaboration is happening, and developers aren't stepping on each other's toes. The place where this is managed and the history and evolution of your code are stored is your SCM system.

Popular SCM systems today include Subversion (SVN), Git, Mercurial, and CVS. SCM systems have evolved significantly over time, but switching from one type of SCM to another isn't always easy. This is because if all of your code and history are in one SCM system, moving all of it and asking your development team to start using a new system usually takes quite a bit of effort. There are tools to help you migrate from one system to another, and following those tools leads you down the path of the evolution of source control. For example, a tool called cvs2svn will help you migrate from CVS to the newer and more popular SVN. Google "migrating SVN to CVS"; most of the links will point you in the other direction. There are also tools that help you migrate from SVN to the newer Git or Mercurial, but going in the opposite direction isn't so common.

If you're not using SCM for your project, I strongly recommend you start using it. Even teams of one developer benefit from saving the history of their codebase over time; if you make a mistake or need to restore some functionality that you took out, SCM makes it easy to go back in time and bring old code that you got rid of back to life.

4.2. Preparing for analysis: generate the richest set of data you can

The most obvious data you can get from these systems is how much code is changing in CLOC and how much individuals on the team are changing it. As long as you're using source control, you have that data. But if you want to answer more interesting questions like the ones I posed at the beginning of the chapter: